Rapport de TP

TP 3 : Architecture distribuée

BOUHAMIDI-ALAOUI Khaoula

Filière : IACS1

Encadrant : Pr. BE ELBAGHAZAOUI

Établissement : ENSA BM

Table des matières

1 Introduction **3**

2 Technologies utilisées **3**

3 Architecture cible du système **3**

4 Structure du projet & prérequis **4**
4.1 Prérequis à installer 4
4.2 Création de la structure de dossiers 5

5 Docker Compose — Lancer tous les composants **6**

6 Configuration Nginx — Load Balancer **7**

7 Services Node.js — MongoDB, Redis et RabbitMQ **7**
7.1 Connexion MongoDB 7
7.2 Connexion Redis 8
7.3 Connexion RabbitMQ 9

8 Routes API — POST /commande et GET /commandes **10**

9 Serveur principal et consommateur RabbitMQ **11**
9.1 Serveur principal 11
9.2 Consommateur RabbitMQ 12
9.3 Dockerfile 13

10 Lancement du système complet **14**

11 Vérification des services actifs **15**

12 Tests avec Postman — Scénario complet **16**
12.1 Test 1 : Créer une commande (POST) 16
12.2 Test 2 : Lister les commandes (GET) — Observer le cache 17
12.3 Test 3 : Créer plusieurs commandes et observer le load balancing 19

13 Questions de compte-rendu **20**

14 Conclusion **21**

1 Introduction

Ce TP a pour objectif de déployer une architecture distribuée complète avec plusieurs composants interconnectés. Le système permet de gérer des commandes à l'aide d'un client HTTP, d'un load balancer, de deux instances du serveur applicatif, d'une base de données MongoDB, d'un cache Redis et d'un bus de messages RabbitMQ.

Le but principal est de comprendre le rôle de chaque composant dans le système distribué et de suivre le flux complet suivant : création d'une commande, stockage dans MongoDB, mise en cache via Redis puis publication d'un événement dans RabbitMQ.

Commentaire

Ce TP nous a permis de manipuler concrètement les notions essentielles d'architecture distribuée, notamment la répartition de charge, la persistance des données, le cache et la communication asynchrone.

2 Technologies utilisées

Les technologies utilisées dans ce TP sont les suivantes :

- Node.js / Express
- MongoDB
- Redis
- RabbitMQ
- Nginx
- Docker / Docker Compose
- Postman
- Visual Studio Code

Commentaire

Chaque technologie remplit un rôle précis dans le système. Node.js gère la logique métier, MongoDB stocke les commandes, Redis améliore les performances en jouant le rôle de cache, RabbitMQ assure l'échange de messages, tandis que Nginx répartit les requêtes entre les instances du serveur.

3 Architecture cible du système

L'architecture du système se compose des éléments suivants :

- **Client :** Postman
- **Load Balancer :** Nginx
- **Serveur applicatif :** Node.js / Express
- **Base de données :** MongoDB
- **Cache :** Redis
- **Bus de messages :** RabbitMQ

Le flux de données est le suivant :

Postman → Nginx :80 → Node.js :3001 / :3002
Node.js → MongoDB

Node.js → Redis
Node.js → RabbitMQ → Consommateur

```
mkdir tp-archi-distribuee
cd tp-archi-distribuee

tp-archi-distribuee/
├── docker-compose.yml      ← orchestre tous les conteneurs
├── nginx/
│   └── nginx.conf          ← configuration du load balancer
├── app/
```

```
│   ├── package.json
│   ├── server.js           ← serveur Node.js principal
│   ├── routes/
│   │   └── commandes.js    ← routes API
│   ├── services/
│   │   ├── mongodb.js      ← connexion MongoDB
│   │   ├── redis.js        ← connexion Redis
│   │   └── rabbitmq.js     ← connexion RabbitMQ
│   └── consumer.js         ← consommateur de messages
```

FIGURE 1 – Architecture cible du système

Commentaire

Cette architecture permet de séparer clairement les responsabilités entre les composants et montre l'intérêt d'un système distribué pour améliorer la performance, l'organisation et la résilience.

4 Structure du projet & prérequis

4.1 Prérequis à installer

Avant de commencer le TP, nous avons utilisé les outils suivants :

- Node.js
- Docker Desktop
- Postman
- Visual Studio Code

Commentaire

Ces outils étaient indispensables pour développer l'application, exécuter les conteneurs Docker, envoyer des requêtes HTTP et vérifier le fonctionnement global du système.

4.2 Création de la structure de dossiers

Nous avons créé un dossier principal contenant les éléments suivants :
- docker-compose.yml
- le dossier nginx/
- le dossier app/

Le dossier app/ contient les fichiers principaux du backend :
- server.js
- consumer.js
- package.json
- package-lock.json
- Dockerfile
- routes/commandes.js
- services/mongodb.js
- services/redis.js
- services/rabbitmq.js

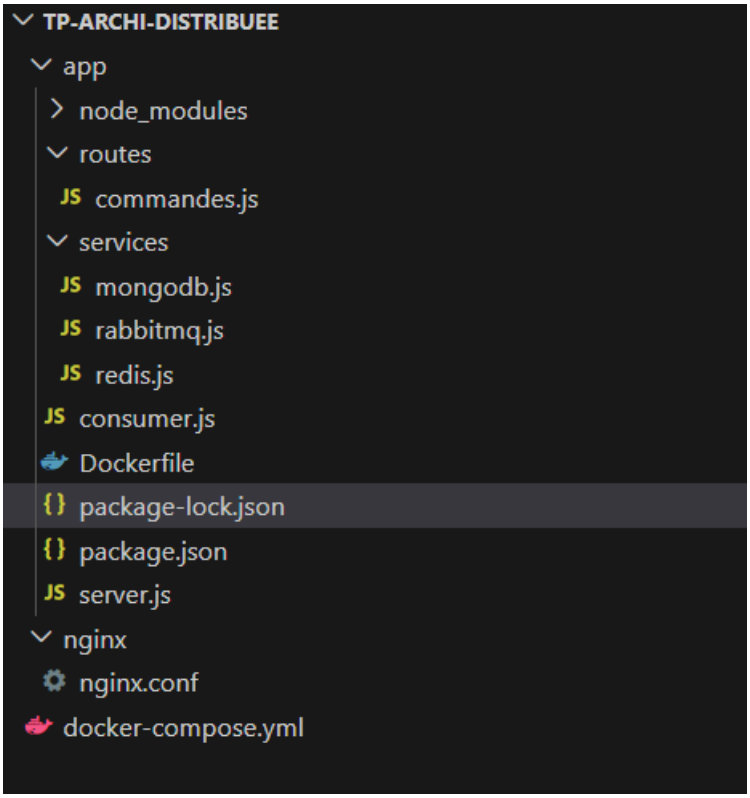


FIGURE 2 – Structure du projet

Commentaire

L'organisation du projet en sous-dossiers permet de séparer les rôles de chaque fichier. Cela rend le code plus clair, plus structuré et plus facile à maintenir.

5 Docker Compose — Lancer tous les composants

Dans cette étape, nous avons créé le fichier docker-compose.yml afin d'orchestrer tous les services du système. Ce fichier permet de démarrer MongoDB, Redis, RabbitMQ, les deux instances Node.js, le consommateur et Nginx à partir d'une seule commande.

Contenu du fichier docker-compose.yml

docker-compose.yml × | Codex Diff | package.json | package-lock.json | Docker

docker-compose.yml

```yml
services:
  mongodb:
    image: mongo:6
    container_name: mongodb
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 15s

  redis:
    image: redis:7-alpine
    container_name: redis
    ports:
      - "6379:6379"
    healthcheck:
```

FIGURE 3 – Fichier docker-compose.yml

Commentaire

Le fichier docker-compose.yml est au centre de l'architecture. Il permet de définir chaque service, ses ports, ses dépendances et les commandes nécessaires à son exécution.

6 Configuration Nginx — Load Balancer

Nginx a été configuré comme load balancer afin de répartir les requêtes entre les deux instances du serveur applicatif. L'algorithme utilisé est le **Round-robin**.

Contenu du fichier nginx.conf

```
nginx > nginx.conf
events {
    worker_connections 1024;
}

http {
    upstream backend {
        server app1:3001;
        server app2:3002;
    }

    server {
        listen 80;

        location / {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

FIGURE 4 – Configuration Nginx

Commentaire

Nginx reçoit les requêtes du client en premier puis les envoie vers l'une des deux instances Node.js. Cette configuration permet une meilleure répartition de charge sur le système.

7 Services Node.js — MongoDB, Redis et RabbitMQ

7.1 Connexion MongoDB

Nous avons créé un service MongoDB afin de gérer la connexion à la base de données ainsi que le modèle Commande.

Contenu du fichier mongodb.js

```
app > services > JS mongodb.js > connectMongo
const mongoose = require('mongoose');

const CommandeSchema = new mongoose.Schema({
  produit: String,
  quantite: Number,
  client: String,
  statut: { type: String, default: 'en_attente' },
  date: { type: Date, default: Date.now },
  dateTraitement: { type: Date, default: null }
});

const Commande = mongoose.model('Commande', CommandeSchema);

async function connectMongo() {
  const mongoUri = process.env.MONGODB_URI || 'mongodb://mongodb:27017/boutique';
  await mongoose.connect(mongoUri);
  console.log('MongoDB connecte');
}

module.exports = { connectMongo, Commande };
```

FIGURE 5 – Service MongoDB

Commentaire

Ce fichier assure la connexion à MongoDB et définit la structure des données représentant une commande. Il constitue la couche de persistance de l'application.

7.2 Connexion Redis

Nous avons créé un service Redis afin de mettre en cache les résultats des requêtes GET.

Contenu du fichier redis.js

```
docker-compose.yml | Codex Diff | {} package.json | {} package-lock.json
app > services > JS redis.js > ...
const Redis = require('ioredis');

const redis = new Redis({
  host: process.env.REDIS_HOST || 'redis',
  port: Number(process.env.REDIS_PORT || 6379),
  maxRetriesPerRequest: null
});

redis.on('connect', () => console.log('Redis connecte'));

module.exports = redis;
```

FIGURE 6 – Service Redis

Commentaire

Redis permet de réduire le nombre d'accès à la base MongoDB en mémorisant temporairement les données les plus récentes. Cela améliore la rapidité de réponse du système.

7.3 Connexion RabbitMQ

Nous avons également créé un service RabbitMQ afin de publier un message après chaque création de commande.

Contenu du fichier rabbitmq.js

app > services > JS rabbitmq.js > connectRabbitMQ

```javascript
const amqp = require('amqplib');

let channel;
const QUEUE = 'commandes';

async function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function connectRabbitMQ() {
  const rabbitUri = process.env.RABBITMQ_URI || 'amqp://admin:admin@rabbitmq:5672';
  const retries = Number(process.env.RABBITMQ_RETRIES || 10);
  const delay = Number(process.env.RABBITMQ_RETRY_DELAY_MS || 3000);
  let lastError;

  for (let attempt = 1; attempt <= retries; attempt += 1) {
    try {
      const conn = await amqp.connect(rabbitUri);
      channel = await conn.createChannel();
      await channel.assertQueue(QUEUE, { durable: true });
      console.log('RabbitMQ connecte');
```

```javascript
      console.log('RabbitMQ connecte');
      return;
    } catch (error) {
      lastError = error;
      console.warn(`Tentative RabbitMQ ${attempt}/${retries} echouee: ${error.message}`);
      if (attempt < retries) {
        await wait(delay);
      }
    }
  }

  throw lastError;
}

```

```
function publierCommande(commande) {
  if (!channel) {
    throw new Error('Canal RabbitMQ indisponible');
  }

  const msg = JSON.stringify(commande);
  channel.sendToQueue(QUEUE, Buffer.from(msg), { persistent: true });
  console.log('Message envoye a RabbitMQ');
}

function getQueueName() {
  return QUEUE;
}

module.exports = { connectRabbitMQ, publierCommande, getQueueName };
```

FIGURE 7 – Service RabbitMQ

Commentaire

RabbitMQ joue le rôle de bus de messages. Il permet à l'application de publier des événements sans attendre immédiatement leur traitement, ce qui rend le système plus souple et plus robuste.

8 Routes API — POST /commande et GET /commandes

Dans cette partie, nous avons développé les routes principales qui permettent de créer une commande et de récupérer la liste des commandes.

Contenu du fichier commandes.js

```
app > routes > JS commandes.js > router.get('/') callback
const express = require('express');
const router = express.Router();
const { Commande } = require('../services/mongodb');
const redis = require('../services/redis');
const { publierCommande } = require('../services/rabbitmq');

const INSTANCE = process.env.INSTANCE || 'Instance-?';
const TTL = Number(process.env.REDIS_TTL || 30);

function getCacheKey(id) {
  return `commande:${id}`;
}

router.post('/', async (req, res) => {
  try {
    const { produit, quantite, client } = req.body;

    if (!produit || !client || !Number.isInteger(quantite) || quantite <= 0) {
      return res.status(400).json({ erreur: 'Champs invalides' });
    }
```

```
22      const commande = new Commande({ produit, quantite, client });
23      await commande.save();
24
25      await redis.del('toutes_commandes', getCacheKey(commande._id));
26
27      publierCommande({
28        id: commande._id.toString(),
29        produit,
30        quantite,
31        client
32      });
33
34      res.status(201).json({
35        message: 'Commande creee',
36        commande,
37        traitee_par: INSTANCE
38      });
39    } catch (err) {
40      res.status(500).json({ erreur: err.message });
41    }
```

FIGURE 8 – Routes API des commandes

Commentaire

La route POST permet de créer une nouvelle commande, de l'enregistrer dans MongoDB, d'invalider le cache Redis puis d'envoyer un message dans RabbitMQ. La route GET permet de retourner les commandes soit depuis MongoDB, soit depuis Redis si le cache est disponible.

9 Serveur principal et consommateur RabbitMQ

9.1 Serveur principal

Le serveur principal initialise Express, charge les routes, connecte MongoDB et RabbitMQ puis lance l'application sur le port prévu.

Contenu du fichier server.js

```
app > JS server.js > start
1   const express = require('express');
2   const { connectMongo } = require('./services/mongodb');
3   const { connectRabbitMQ } = require('./services/rabbitmq');
4   const commandesRoutes = require('./routes/commandes');
5
6   const app = express();
7   const PORT = process.env.PORT || 3001;
8   const INSTANCE = process.env.INSTANCE || 'Instance-?';
9
10  app.use(express.json());
11  app.use('/commandes', commandesRoutes);
12
13  app.get('/health', (req, res) => {
14    res.json({ status: 'OK', instance: INSTANCE });
15  });
```

```
15  });
16
17  async function start() {
18    try {
19      await connectMongo();
20      await connectRabbitMQ();
21
22      app.listen(PORT, () => {
23        console.log(`${INSTANCE} lancee sur le port ${PORT}`);
24      });
25    } catch (error) {
26      console.error(`Echec du demarrage de ${INSTANCE}:`, error.message);
27      process.exit(1);
28    }
29  }
30
31  start();
```

FIGURE 9 – Serveur principal

Commentaire

Le fichier server.js est le point d'entrée de l'application. Il permet de démarrer le backend et d'assurer la liaison entre les différents services du système.

9.2 Consommateur RabbitMQ

Nous avons mis en place un consommateur indépendant chargé de lire les messages publiés dans RabbitMQ et d'afficher une notification dans la console.

Contenu du fichier consumer.js

```
app > JS consumer.js > connectConsumerRabbitMQ
1   const amqp = require('amqplib');
2   const { connectMongo, Commande } = require('./services/mongodb');
3   const redis = require('./services/redis');
4   const { getQueueName } = require('./services/rabbitmq');
5
6   async function wait(ms) {
7     return new Promise((resolve) => setTimeout(resolve, ms));
8   }
9
10  async function connectConsumerRabbitMQ() {
11    const rabbitUri = process.env.RABBITMQ_URI || 'amqp://admin:admin@rabbitmq:5672';
12    const retries = Number(process.env.RABBITMQ_RETRIES || 10);
13    const delay = Number(process.env.RABBITMQ_RETRY_DELAY_MS || 3000);
14    let lastError;
15
16    for (let attempt = 1; attempt <= retries; attempt += 1) {
17      try {
18        return await amqp.connect(rabbitUri);
19      } catch (error) {
20        lastError = error;
21        console.warn(`Tentative consumer RabbitMQ ${attempt}/${retries} echouee: ${error.message}`);
```

```
22          if (attempt < retries) {
23            await wait(delay);
24          }
25        }
26      }
27
28      throw lastError;
29    }
30
31    async function startConsumer() {
32      await connectMongo();
33
34      const conn = await connectConsumerRabbitMQ();
35      const channel = await conn.createChannel();
36      const queue = getQueueName();
37
38      await channel.assertQueue(queue, { durable: true });
39      channel.prefetch(1);
40
41      console.log('Consumer en attente des messages...');
```

FIGURE 10 – Consommateur RabbitMQ

Commentaire

Le consommateur simule un microservice indépendant chargé de traiter les messages. Cette séparation permet d'introduire une vraie logique de communication asynchrone.

9.3 Dockerfile

Le Dockerfile a été utilisé pour construire l'image de l'application Node.js.

Contenu du fichier Dockerfile

```
app > Dockerfile
1   FROM node:20-alpine
2
3   WORKDIR /app
4
5   COPY package*.json ./
6   RUN npm install
7
8   COPY . .
9
10  CMD ["node", "server.js"]
11
```

FIGURE 11 – Dockerfile de l'application

Commentaire

Le Dockerfile permet de préparer automatiquement l'environnement d'exécution de l'application dans un conteneur, ce qui facilite son déploiement.

10 Lancement du système complet

Après la configuration des différents fichiers, nous avons lancé l'ensemble du système à l'aide de la commande suivante :

```
docker-compose up --build
```

Listing 10 – Commande de lancement

Les logs affichés dans le terminal ont permis de vérifier le bon démarrage de MongoDB, Redis, RabbitMQ, des deux instances Node.js, du consommateur et de Nginx.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\tp-archi-distribuee> docker-compose up --build
#16 DONE 0.0s
[+] up 6/6
 ✓ Image tp-archi-distribuee-app1     Built
 ✓ Image tp-archi-distribuee-app2     Built
 ✓ Image tp-archi-distribuee-consumer Built
 ✓ Container app1                     Recreated
 ✓ Container consumer                 Recreated
 ✓ Container app2                     Recreated
Attaching to app1, app2, consumer, mongodb, nginx, rabbitmq, redis
Container rabbitmq Waiting
Container mongodb Waiting
Container redis Waiting
Container rabbitmq Waiting
Container mongodb Waiting
Container mongodb Waiting
Container redis Waiting
Container redis Waiting
Container rabbitmq Waiting
redis  | 1:C 03 Apr 2026 10:41:00.656 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
redis  | 1:C 03 Apr 2026 10:41:00.656 * Redis version=7.4.8, bits=64, commit=00000000, modified=0, pid=1, just started
redis  | 1:C 03 Apr 2026 10:41:00.656 # Warning: no config file specified, using the default config. In order to specify a config file use
nf
redis  | 1:M 03 Apr 2026 10:41:00.657 * monotonic clock: POSIX clock_gettime
redis  | 1:M 03 Apr 2026 10:41:00.662 * Running mode=standalone, port=6379.
redis  | 1:M 03 Apr 2026 10:41:00.663 * Server initialized
redis  | 1:M 03 Apr 2026 10:41:00.664 * Loading RDB produced by version 7.4.8
redis  | 1:M 03 Apr 2026 10:41:00.664 * RDB age 76 seconds
redis  | 1:M 03 Apr 2026 10:41:00.664 * RDB memory usage when created 1.05 Mb
redis  | 1:M 03 Apr 2026 10:41:00.664 * Done loading RDB, keys loaded: 0, keys expired: 0.
redis  | 1:M 03 Apr 2026 10:41:00.664 * DB loaded from disk: 0.000 seconds
redis  | 1:M 03 Apr 2026 10:41:00.664 * Ready to accept connections tcp
```

```
6","uuid":"0275af04-fafa-46dc-a3d9-6eeee07e4f5f","connectionId":2,"connectionCount":0}}
Container mongodb Healthy
Container mongodb Healthy
Container mongodb Healthy  o View Config   w Enable Watch   d Detach
Container rabbitmq Healthy
Container rabbitmq Healthy
Container rabbitmq Healthy
Container redis Healthy
Container redis Healthy
Container redis Healthy
Container app2 Waiting
Container app1 Waiting
consumer  |
```

FIGURE 12 – Lancement complet du système

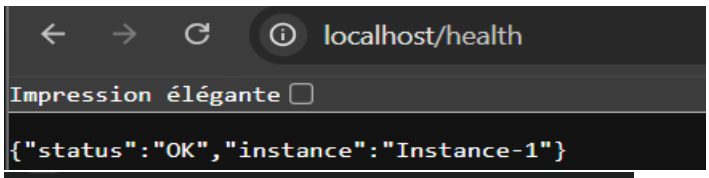
Commentaire

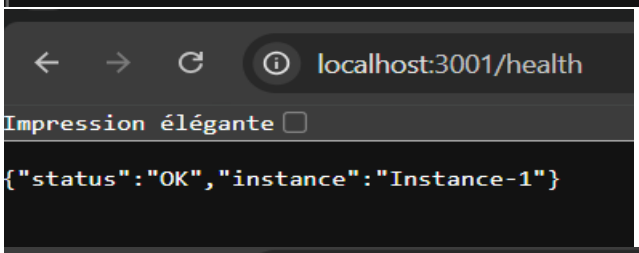
Cette étape confirme que tous les services communiquent correctement entre eux et que l'architecture distribuée est bien opérationnelle.

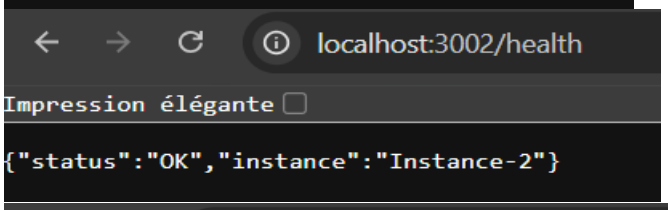
11 Vérification des services actifs

Afin de vérifier que tous les composants étaient bien accessibles, nous avons testé les adresses suivantes :

- http://localhost/health
- http://localhost:3001/health
- http://localhost:3002/health
- http://localhost:15672







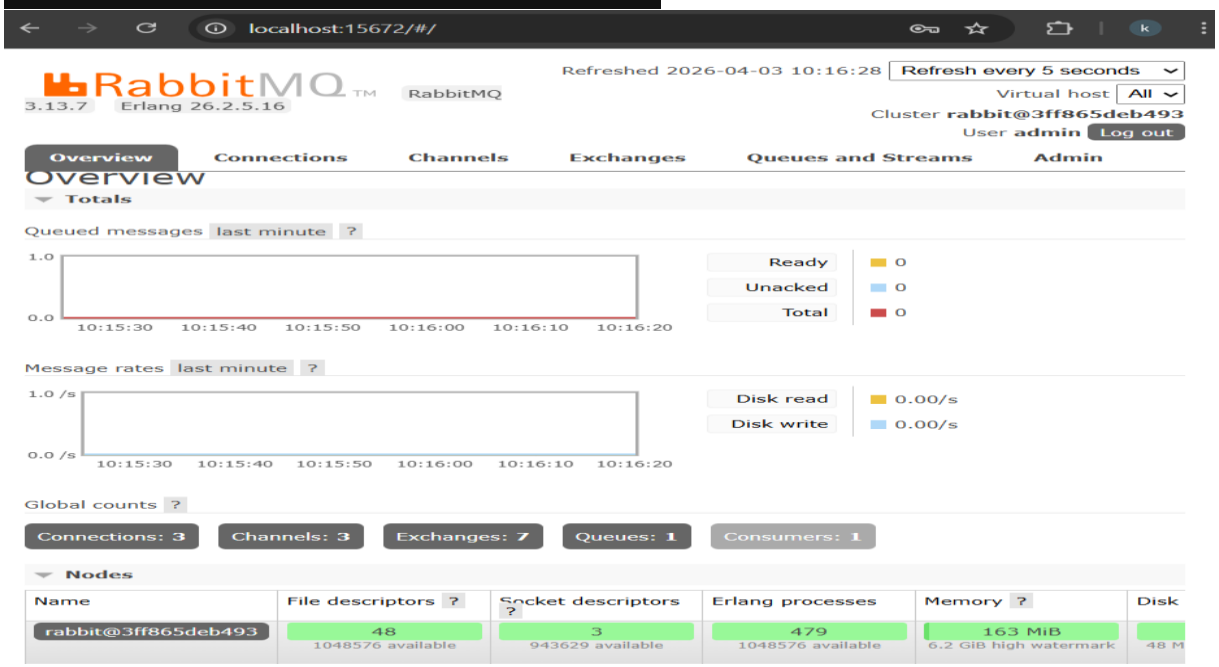


FIGURE 13 – Vérification des services actifs

Commentaire

Cette vérification permet de confirmer que Nginx fonctionne bien comme point d'entrée du système et que les deux instances applicatives répondent correctement.

12 Tests avec Postman — Scénario complet

12.1 Test 1 : Créer une commande (POST)

Nous avons utilisé Postman pour envoyer une requête POST vers l'URL suivante :

http://localhost/commandes

Le corps de la requête contient les informations de la commande au format JSON.

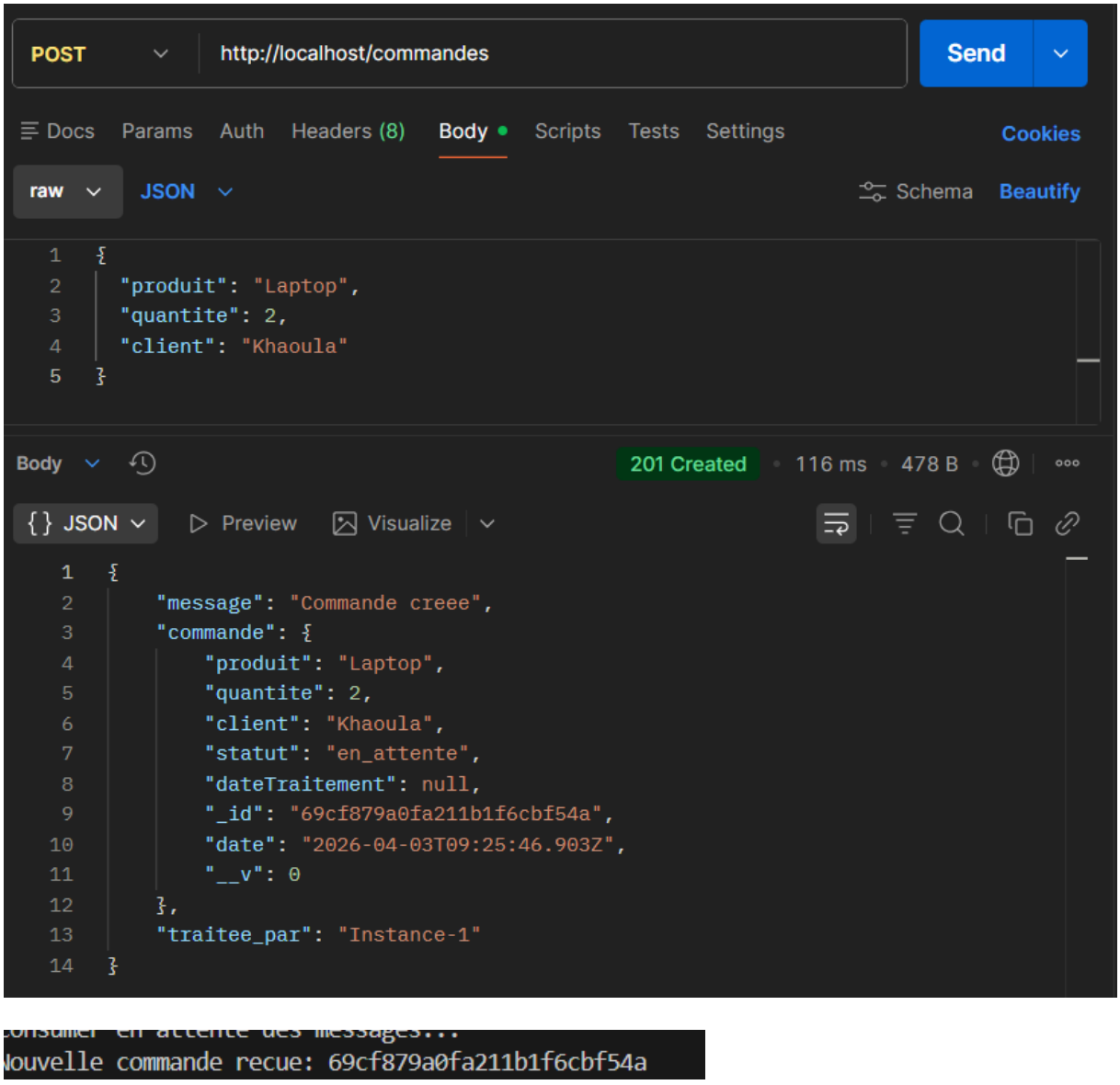


FIGURE 14 – Test POST /commandes

Commentaire

Ce test permet de vérifier que la commande est bien reçue par le serveur, validée puis enregistrée dans MongoDB. Il permet aussi de confirmer qu'un message est publié dans RabbitMQ après la création de la commande.

12.2 Test 2 : Lister les commandes (GET) — Observer le cache

Nous avons ensuite envoyé deux requêtes GET successives vers :

http://localhost/commandes

Le premier appel récupère les données depuis MongoDB. Le second appel, s'il est effectué avant expiration du cache, retourne les données depuis Redis.

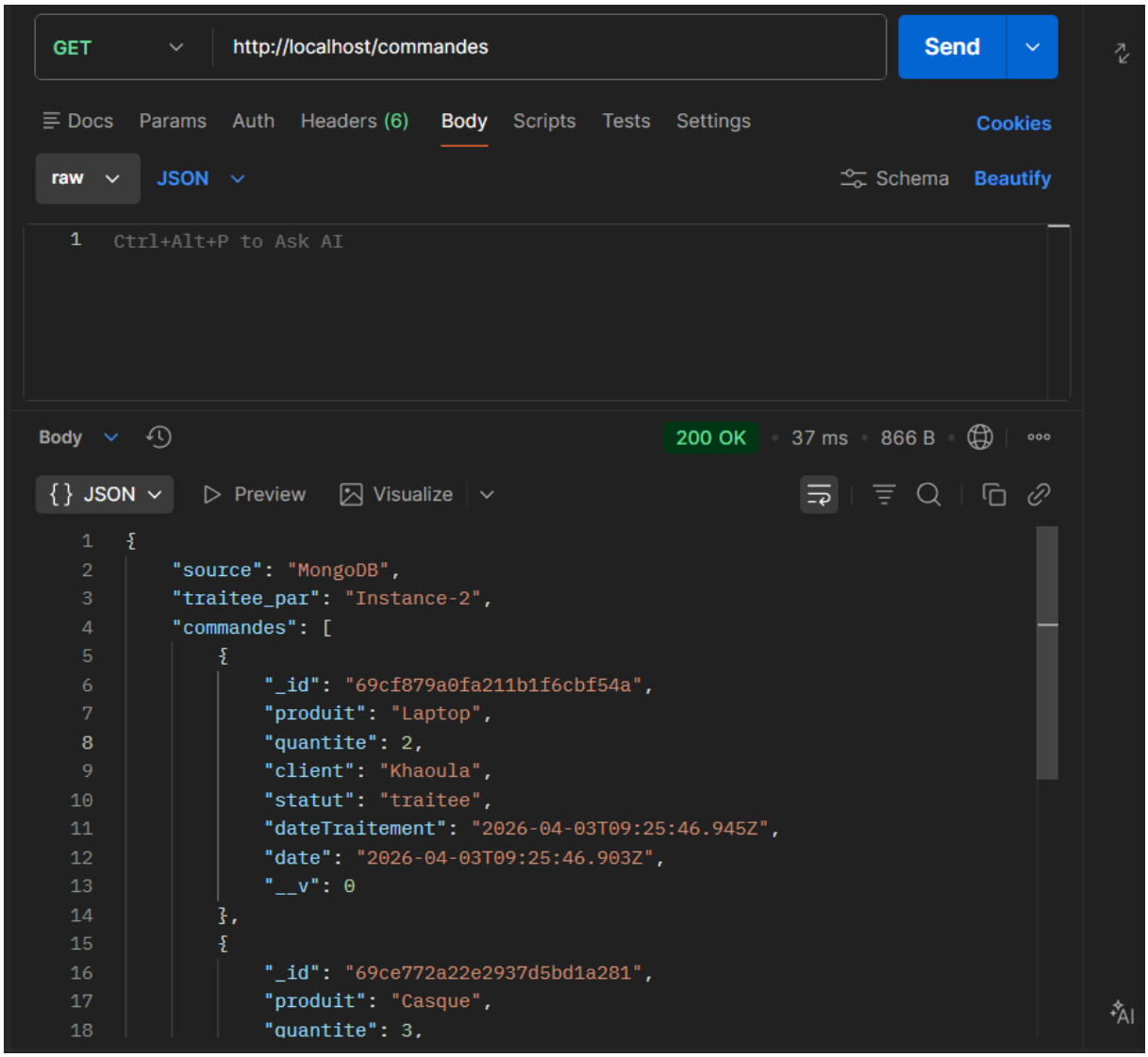


FIGURE 15 – Premier GET /commandes — Source MongoDB

Commentaire

Le premier GET interroge directement MongoDB car le cache Redis n'est pas encore alimenté. Ce comportement est normal lors du premier accès aux données.

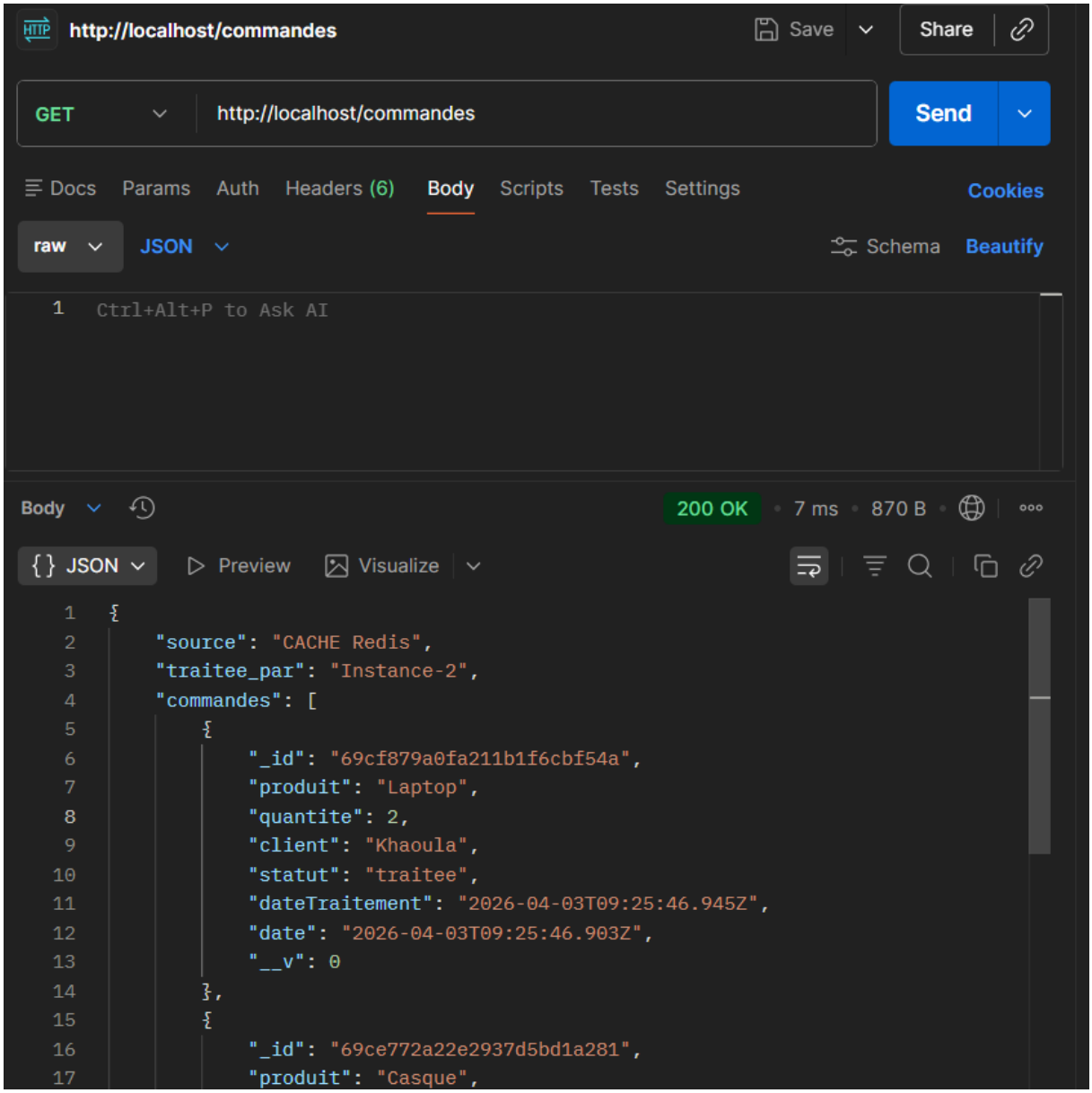


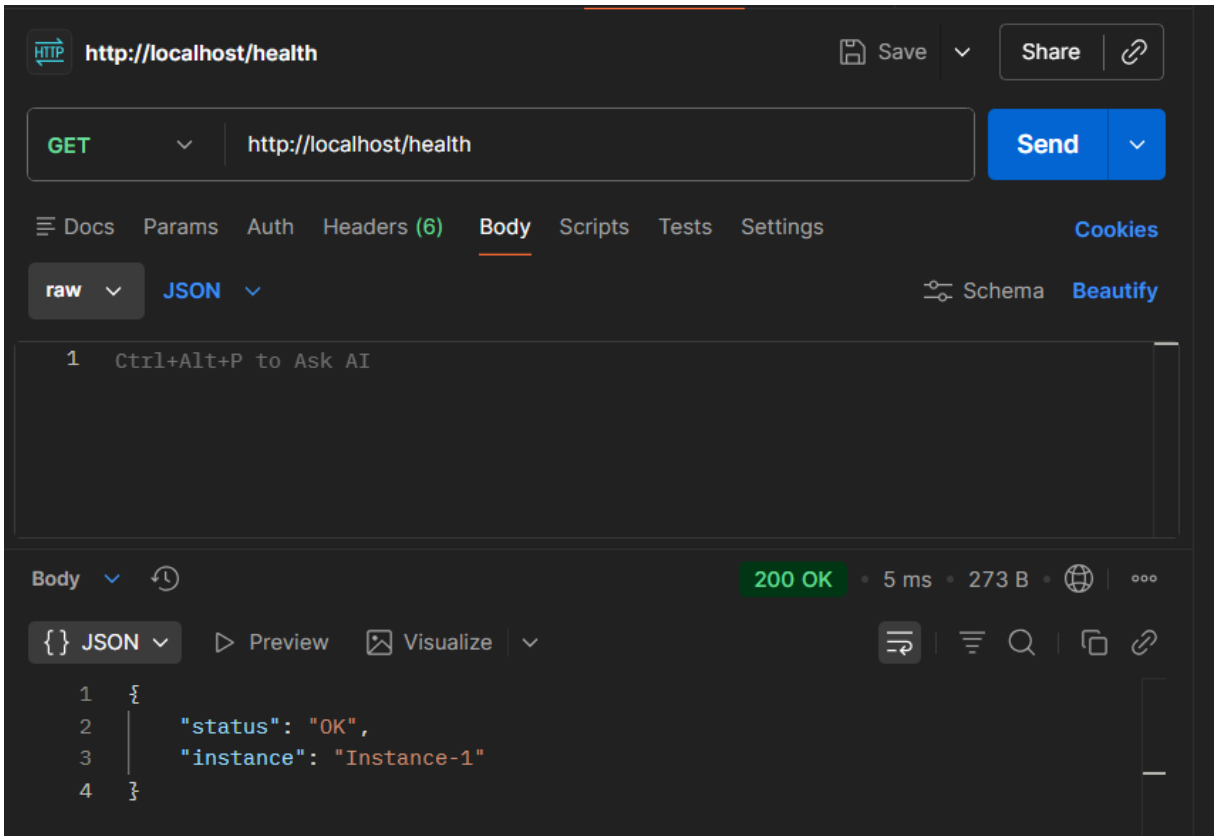
FIGURE 16 – Deuxième GET /commandes — Source Redis

Commentaire

Le deuxième GET retourne les données depuis Redis, ce qui montre que le cache fonctionne correctement. Cela permet d'obtenir une réponse plus rapide et d'éviter une nouvelle lecture depuis MongoDB.

12.3 Test 3 : Créer plusieurs commandes et observer le load balancing

Enfin, nous avons envoyé plusieurs requêtes successives afin d'observer l'alternance entre Instance-1 et Instance-2 dans le champ traitee_par ou instance.



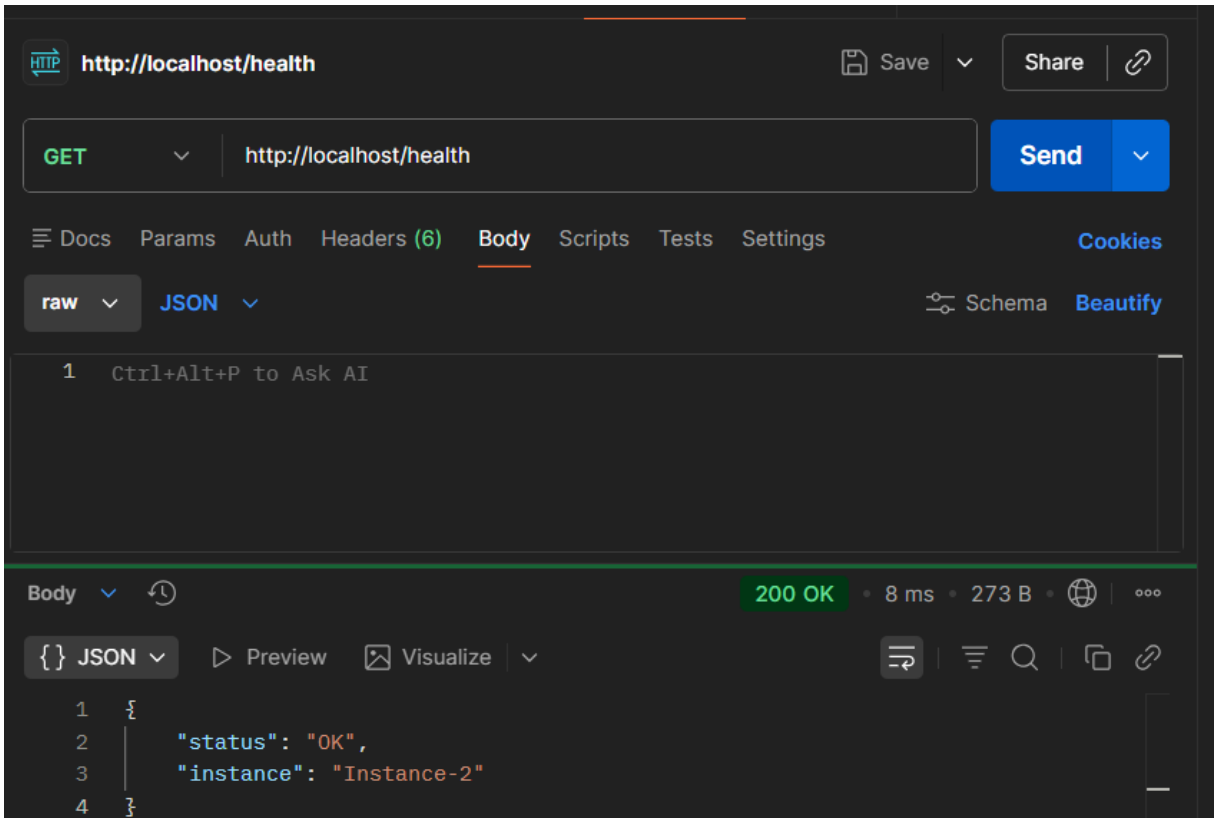


FIGURE 17 – Observation du load balancin

Commentaire

Ce test montre que Nginx répartit correctement les requêtes entre les deux instances Node.js. L'alternance observée confirme l'utilisation de l'algorithme Round-robin.

13 Questions de compte-rendu

Q1. Quel composant reçoit en premier les requêtes du client ? Quel algorithme utilise-t-il ?

Le premier composant qui reçoit les requêtes du client est **Nginx**, qui joue le rôle de load balancer. Il utilise par défaut l'algorithme **Round-robin**, ce qui signifie qu'il répartit les requêtes en alternance entre les instances disponibles.

Q2. Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ? Pourquoi ?

Si app1 tombe en panne, Nginx continue à fonctionner tant qu'une autre instance, comme app2, reste disponible. Le load balancer peut alors rediriger les requêtes vers l'instance active, ce qui améliore la disponibilité du système.

Q3. Expliquez la différence observée entre le 1er GET (source MongoDB) et le 2ème GET (source Redis).

Lors du premier GET, les données sont récupérées directement depuis MongoDB car le cache Redis n'a pas encore été rempli. Lors du deuxième GET, les données sont lues depuis Redis, ce qui rend la réponse plus rapide.

Q4. Pourquoi invalide-t-on le cache Redis après un POST /commande ?

Après l'ajout d'une nouvelle commande, les données présentes dans Redis risquent de devenir obsolètes. Il faut donc invalider le cache afin que le prochain GET récupère des données actualisées depuis MongoDB.

Q5. En quoi le bus de messages RabbitMQ améliore-t-il la résilience du système ?

RabbitMQ améliore la résilience car il permet de découpler le traitement principal de l'application des tâches secondaires, comme les notifications. Même si le consommateur n'est pas immédiatement disponible, le message peut rester dans la file puis être traité plus tard.

Q6. Si on voulait ajouter une 3ème instance Node.js, quelle ligne faudrait-il modifier dans nginx.conf ?

Il faudrait modifier le bloc upstream backend dans nginx.conf en ajoutant une nouvelle ligne :

```
server app3:3003;
```

Listing 11 – Ajout d'une 3ème instance dans nginx.conf

Commentaire

Cette modification permettrait à Nginx de répartir les requêtes entre trois serveurs applicatifs au lieu de deux.

14 Conclusion

Ce TP nous a permis de comprendre concrètement le fonctionnement d'une architecture distribuée moderne. Nous avons appris à mettre en place plusieurs services interconnectés, à répartir les requêtes avec Nginx, à utiliser MongoDB pour la persistance, Redis pour le cache et RabbitMQ pour la communication asynchrone.

Les tests réalisés avec Postman et les logs observés dans le terminal ont confirmé le bon fonctionnement global du système ainsi que l'intérêt de cette architecture en termes de performance, de modularité et de résilience.